arXiv:2401.02416v3 [cs.CV] 25 Jun 2024

# ODIN: A Single Model for 2D and 3D Segmentation

Ayush Jain[1], Pushkal Katara[1], Nikolaos Gkanatsios[1], Adam W. Harley[2],
Gabriel Sarch[1], Kriti Aggarwal[3], Vishrav Chaudhary[3], Katerina Fragkiadaki[1]
[1]Carnegie Mellon University, [2]Stanford University, [3] Microsoft
{ayushj2, pkatara ngkanats,gsarch,kfragki2}@andrew.cmu.edu
aharley@cs.stanford.edu, {kragga,vchaudhary}@microsoft.com

## Abstract

*State-of-the-art models on contemporary 3D segmentation benchmarks like ScanNet consume and label dataset-provided 3D point clouds, obtained through post processing of sensed multiview RGB-D images. They are typically trained in-domain, forego large-scale 2D pre-training and outperform alternatives that featurize the posed RGB-D multiview images instead. The gap in performance between methods that consume posed images versus post-processed 3D point clouds has fueled the belief that 2D and 3D perception require distinct model architectures. In this paper, we challenge this view and propose ODIN (Omni-Dimensional INstance segmentation), a model that can segment and label both 2D RGB images and 3D point clouds, using a transformer architecture that alternates between 2D within-view and 3D cross-view information fusion. Our model differentiates 2D and 3D feature operations through the positional encodings of the tokens involved, which capture pixel coordinates for 2D patch tokens and 3D coordinates for 3D feature tokens. ODIN achieves state-of-the-art performance on ScanNet200, Matterport3D and AI2THOR 3D instance segmentation benchmarks, and competitive performance on ScanNet, S3DIS and COCO. It outperforms all previous works by a wide margin when the sensed 3D point cloud is used in place of the point cloud sampled from 3D mesh. When used as the 3D perception engine in an instructable embodied agent architecture, it sets a new state-of-the-art on the TEACh action-from-dialogue benchmark. Our code and checkpoints can be found at the project website https://odin-seg.github.io.*

## 1. Introduction

There has been a surge of interest in porting 2D foundational image features to 3D scene understanding [8, 14, 21, 23, 37, 40, 46–48]. Some methods lift pre-trained 2D image features using sensed depth to 3D feature clouds [8, 37, 40, 47]. Others distill 2D backbones to differentiable parametric 3D models, e.g., NeRFs, by training them per scene to render 2D feature maps of pre-trained backbones [23, 46]. Despite this effort, and despite the ever-growing power of 2D backbones [4, 53], the state-of-the-art on established 3D segmentation benchmarks such as ScanNet [6] and ScanNet200 [41] *still* consists of models that operate directly in 3D, without any 2D pre-training stage [28, 44]. Given the obvious power of 2D pre-training, why is it so difficult to yield improvements in these 3D tasks?

We observe that part of the issue lies in a key implementation detail underlying these 3D benchmark evaluations. Benchmarks like ScanNet do not actually ask methods to use RGB-D images as input, even though this is the sensor data. Instead, these benchmarks first register all RGB-D frames into a single colored point cloud and reconstruct the scene as cleanly as possible, relying on manually tuned stages for bundle adjustment, outlier rejection and meshing, and ask models to label the *output reconstruction*. While it is certainly viable to scan and reconstruct a room before labelling any of the objects inside, this pipeline is perhaps inconsistent with the goals of embodied vision (and typical 2D vision), which involves dealing with actual sensor data and accounting for missing or partial observations. We therefore hypothesize that method rankings will change, and the impact of 2D pre-training will become evident, if we force the 3D models to take posed RGB-D frames as input rather than pre-computed mesh reconstructions. Our revised evaluation setting also opens the door to new methods, which can train and perform inference in either single-view or multi-view settings, with either RGB or RGB-D sensors.

We propose **O**mni-**D**imensional **IN**stance segmentation (ODIN)[†], a model for 2D and 3D object segmentation and labelling that can parse single-view RGB images and/or multiview posed RGB-D images. As shown in Fig. 1, ODIN alternates between 2D and 3D stages in its architecture,

[†]The Norse god Odin sacrificed one of his eyes for wisdom, trading one mode of perception for a more important one. Our approach sacrifices perception on post-processed meshes for perception on posed RGB-D images.

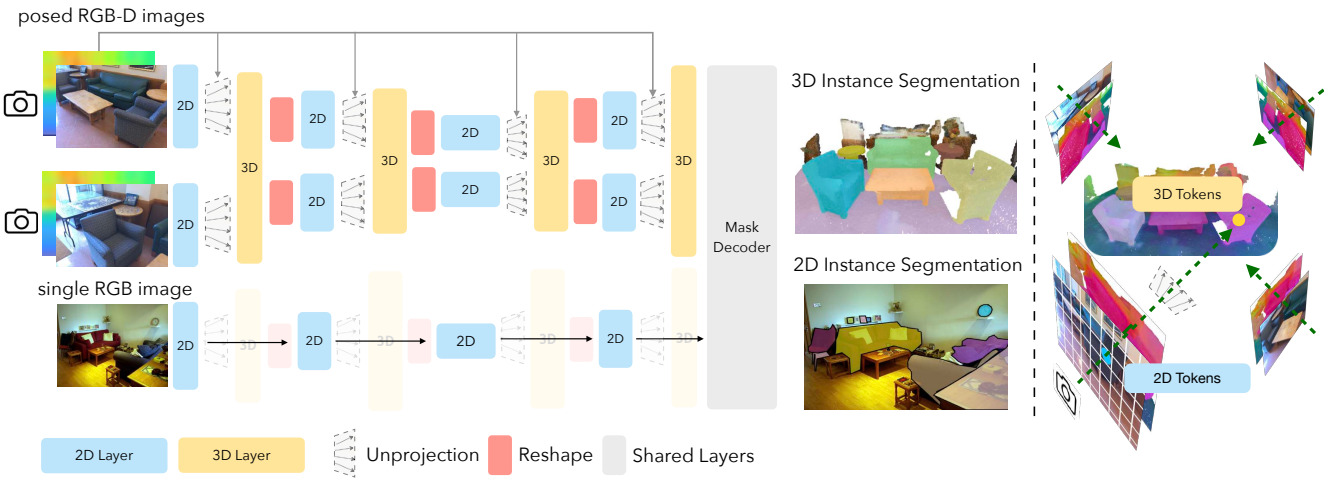


Figure 1. **Omni-Dimensional INstance segmentation (ODIN)** is a model that can parse either a single RGB image or a multiview posed RGB-D sequence into 2D or 3D labelled object segments respectively. **Left**: Given a posed RGB-D sequence as input, ODIN alternates between a within-view 2D fusion and a cross-view 3D fusion. When the input is a single RGB image, the 3D fusion layers are skipped. ODIN shares the majority of its parameters across both RGB and RGB-D inputs, enabling the use of pre-trained 2D backbones. **Right**: At each 2D-to-3D transition, ODIN unprojects 2D feature tokens to their 3D locations using sensed depth and camera intrinsics and extrinsics.

fusing information in 2D within each image view, and in 3D across posed image views. At each 2D-to-3D transition, it unprojects 2D tokens to their 3D locations using the depth maps and camera parameters, and at each 3D-to-2D transition, it projects 3D tokens back to their image locations. Our model differentiates between 2D and 3D features through the positional encodings of the tokens involved, which capture pixel coordinates for 2D patch tokens and 3D coordinates for 3D feature tokens. When dealing with 2D single-view input, our architecture simply skips the 3D layers and makes a forward pass with 2D layers alone.

We test ODIN in 2D and 3D instance segmentation and 3D semantic segmentation on the 2D COCO object segmentation benchmark and the 3D benchmarks of ScanNet [6], ScanNet200 [41], Matterport3D [2], S3DIS [1] and AI2THOR [7, 25]. When compared to methods using pre-computed mesh point cloud as input, our approach performs slightly worse than state-of-the-art on ScanNet and S3DIS, but better on ScanNet200 and Matterport3D. When using real sensor data as input with poses obtained from bundle reconstruction for all methods, our method performs even better, outperforming all prior work by a wide margin, in all datasets. We demonstrate that our model's ability to jointly train on 3D and 2D datasets results in performance increase on 3D benchmarks, and also yields competitive segmentation accuracy on the 2D COCO benchmark. Our ablations show that interleaving 2D and 3D fusion operations outperforms designs where we first process in 2D and then move to 3D, or simply paint 3D points with 2D features. Stepping toward our broader goal of embodied vision, we also deploy ODIN as the 3D object segmentor of a SOTA embodied agent model [42] on the simulation benchmark TEACh [36] in the setup with access to RGB-D and pose information from the simulator, and demonstrate that our model sets a new state-of-the-art. We make our code publicly available at https://odin-seg.github.io.

## 2. Related Work

**3D Instance Segmentation** Early methods in 3D instance segmentation [3, 15, 22, 30, 49, 58] group their semantic segmentation outputs into individual instances. Recently, Mask2Former [4] achieved state-of-the-art in 2D instance segmentation by instantiating *object queries*, each directly predicting an instance segmentation mask by doing dot-product with the feature map of the input image. Inspired by it, Mask3D [44] abandons the grouping strategy of prior 3D models to use the simple decoder head of Mask2Former. MAFT [28] and QueryFormer [34] improve over Mask3D by incorporating better query initialization strategies and/or relative positional embeddings. While this shift to Mask2Former-like architecture brought the 3D instance segmentation architectures closer to their 2D counterparts, the inputs and backbones remain very different: 2D models use pre-trained backbones [16, 33], while 3D methods [44] operate over point clouds and use sparse convolution-based backbones [5], trained from scratch on small-scale 3D datasets. In this work, we propose to directly use RGB-D input and design architectures that can leverage strong 2D backbones to achieve strong performance on 3D benchmarks.

**3D Datasets and Benchmarks** Most 3D models primarily operate on point clouds, avoiding the use of image-based features partly due to the design of popular benchmarks. These benchmarks generate point clouds by processing raw RGB-D sensor data, involving manual and noisy steps that result in misalignments between the reconstructed point cloud and sensor data. For instance, ScanNet [6] undergoes complex mesh reconstruction steps, including bundle reconstruction, implicit TSDF representation fitting, marching cubes, merging and deleting noisy mesh vertices, and finally manual removal of mesh reconstruction with high misalignments. Misalignments introduced by the mesh reconstruction process can cause methods processing sensor data directly to underperform compared to those trained and tested on provided point clouds. Additionally, some datasets, like HM3D [54] lack access to raw RGB-D data. While mesh reconstruction has its applications, many real-time applications need to directly process sensor data.

**2D-based 3D segmentation** Unlike instance segmentation literature, several approaches for semantic segmentation like MVPNet [20], BPNet [17] and DeepViewAgg [40] utilize the sensor point cloud directly instead of the mesh-sampled point cloud. Virtual Multiview Fusion [26] forgoes sensor RGB-D images in favour of rendering RGB-D images from the provided mesh to fight misalignments and low field-of-view in ScanNet images. Similar to our approach, BPNet and DeepViewAgg integrate 2D-3D information at various feature scales and initialize their 2D streams with pre-trained features. Specifically, they employ separate 2D and 3D U-Nets for processing the respective modalities and fuse features from the two streams through a connection module. Rather than employing distinct streams for featurizing raw data, our architecture instantiates a single unified U-Net which interleaves 2D and 3D layers and can handle both 2D and 3D perception tasks with a single unified architecture. Notably, while these works focus solely on semantic segmentation, our single architecture excels in both semantic and instance segmentation tasks.

Recent advancements in 2D foundation models [24, 39] have spurred efforts to apply them to 3D tasks such as point cloud classification [38, 52, 56], zero-shot 3D semantic segmentation [14, 21, 37] and more recently, zero-shot instance segmentation [47]. Commonly, these methods leverage 2D foundation models to featurize RGB images, project 3D point clouds onto these images, employ occlusion reasoning using depth and integrate features from all views through simple techniques like mean-pooling. Notably, these approaches predominantly focus on semantic segmentation, emphasizing pixel-wise labeling, rather than instance labeling, which necessitates cross-view reasoning to associate the same object instance across multiple views. OpenMask3D [47] is the only method that we are aware of that attempts 3D instance segmentation using 2D foundation models, by training a class-agnostic 3D object segmentor on 3D point clouds and labelling it utilizing CLIP features. Despite their effectiveness in a zero-shot setting, they generally lag behind SOTA 3D supervised methods by 15-20%. Rather than relying on features from foundation models, certain works [10, 12] create 3D pseudo-labels using pre-trained 2D models. Another line of work involves fitting Neural-Radiance Fields (NeRFs), incorporating features from CLIP [23, 48] or per-view instance segmentations from state-of-the-art 2D segmentors [46]. These approaches require expensive per-scene optimization that prohibits testing on all test scenes to compare against SOTA 3D discriminative models. Instead of repurposing 2D foundation models for 3D tasks, Omnivore [13] proposes to build a unified architecture that can handle multiple visual modalities like images, videos and single-view RGB-D image but they only show results for classification tasks. We similarly propose a single unified model capable of performing both single-view 2D and multi-view 3D instance and semantic segmentation tasks while utilizing pre-trained weights for the majority of our architecture.

## 3. Method

ODIN's architecture is shown in Fig. 2. It takes either a single RGB image or a set of posed RGB-D images (i.e., RGB images associated with depth maps and camera parameters) and outputs the corresponding 2D or 3D instance segmentation masks and their semantic labels. To achieve this, ODIN alternates between a 2D within-view fusion and a 3D attention-based cross-view fusion, as illustrated in blue blocks and yellow blocks in Fig. 2. A segmentation decoding head predicts instance masks and semantic labels. Notably, ODIN shares the majority of its parameters across both RGB and multiview RGB-D inputs. We detail the components of our architecture below.

**Within-view 2D fusion:** We start from a 2D backbone, such as ResNet50 [16] or Swin Transformer [33], pre-trained for 2D COCO instance segmentation following Mask2Former [4], a state-of-the-art 2D segmentation model. When only a single RGB image is available, we pass it through the full backbone to obtain 2D features at multiple scales. When a posed RGB-D sequence is available, this 2D processing is interleaved with 3D stages, described next. By interleaving within-view and cross-view contextualization, we are able to utilize the pre-trained features from the 2D backbone while also fusing features across views, making them 3D-consistent.

**Cross-view 3D fusion:** The goal of cross-view fusion is to make the individual images' representations consistent across views. As we show in our ablations, cross-view feature consistency is essential for 3D instance segmentation: it enables the segmentation head to realize that a 3D object observed from multiple views is indeed a single instance,

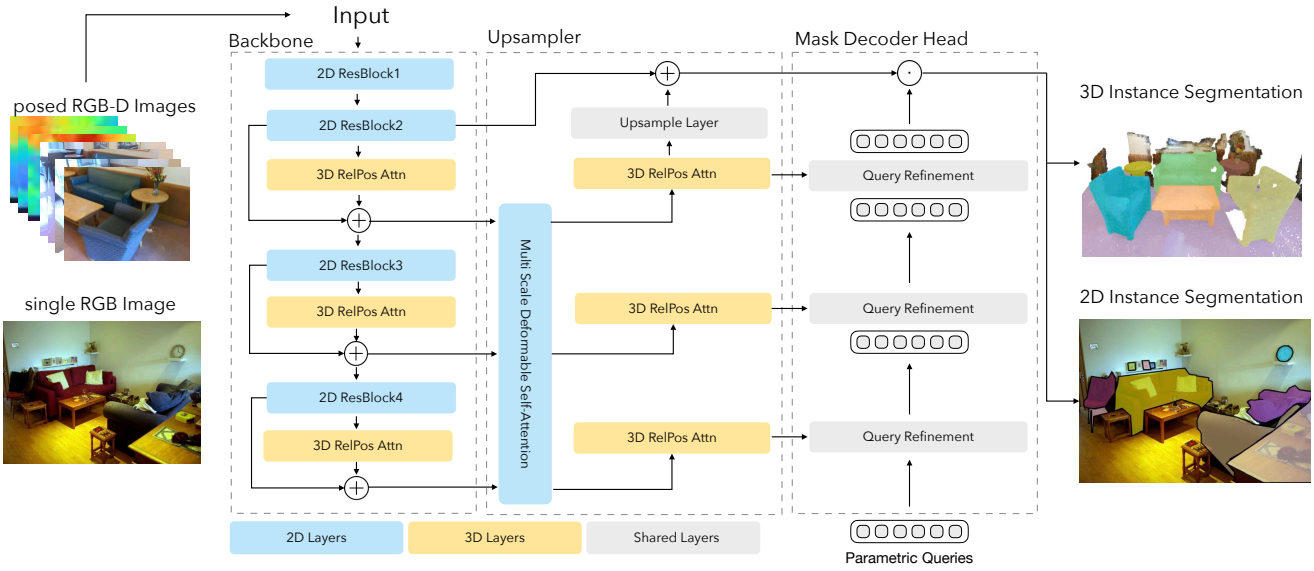


Figure 2. **ODIN Architecture**: The input to our model is either a single RGB image or a multiview RGB-D posed sequence. We feed them to ODIN's backbone which interleaves 2D within-view fusion layers and 3D cross-view attention layers to extract feature maps of different resolutions (scales). These feature maps exchange information through a multi-scale attention operation. Additional 3D fusion layers are used to improve multiview consistency. Then, a mask decoder head is used to initialize and refine learnable slots that attend to the multi-scale feature maps and predict object segments (masks and semantic classes).

rather than a separate instance in each viewpoint.

*1. 2D-to-3D Unprojection:* We unproject each 2D feature map to 3D by lifting each feature vector to a corresponding 3D location, using nearest neighbor depth and known camera intrinsic and extrinsic parameters, using a pinhole camera model. Subsequently, the resulting featurized point cloud undergoes voxelization, where the 3D space is discretized into a volumetric grid. Within each occupied grid cell (voxel), the features and XYZ coordinates are mean-pooled to derive new sets of 3D feature tokens and their respective 3D locations.

*2. 3D $k$-NN Transformer with Relative Positions:* We fuse information across 3D tokens using $k$-nearest-neighbor attention with relative 3D positional embeddings. This is similar to Point Transformers [51, 57], but we simply use vanilla cross-attention instead of the vector attention proposed in those works. Specifically, in our approach, each 3D token attends to its $k$ nearest neighbors. The positional embeddings in this operation are relative to the query token's location. We achieve this by encoding the distance vector between a token and its neighbour with an MLP. The positional embedding for the query is simply encoding of the 0 vector. We therefore have

$$\text{query}_{\text{pos}} = \text{MLP}(0); \tag{1}$$

$$\text{key}_{\text{pos}} = \text{MLP}(p_i - p_j), \tag{2}$$

where $p_i$ represents the 3D tokens, shaped $N \times 1 \times 3$, and $p_j$ represents the $k$ nearest neighbors of each $p_i$, shaped $N \times k \times 3$. In this way, the attention operation is invariant to the absolute coordinates of the 3D tokens and only depends on their relative spatial arrangements. While each 3D token always attends to the same $k$ neighbors, its effective receptive field grows across layers, as the neighbors' features get updated when they perform their own attention [11].

*3. 3D-to-2D Projection:* After contextualizing the tokens in 3D, we project the features back to their original 2D locations. We first copy the feature of each voxel to all points within that voxel. We then reshape these points back into multiview 2D feature maps, so that they may be processed by the next 2D module. The features vectors are unchanged in this transition; the difference lies in their interpretation and shape. In 2D the features are shaped $V \times H \times W \times F$, representing a feature map for each viewpoint, and in 3D they are shaped $N \times F$, representing a unified feature cloud, where $N = V \cdot H \cdot W$.

**Cross-scale fusion and upsampling:** After multiple single-view and cross-view stages, we have access to multiple features maps per image, at different resolutions. We merge these with the help of deformable 2D attention, akin to Mask2Former [4], operating on the three lowest-resolution scales $(1/32, 1/16, 1/8)$. When we have 3D input, we apply an additional 3D fusion layer at each scale after the deformable attention, to restore the 3D consistency. Finally, we use a simple upsampling layer on the $1/8$ resolution feature map to bring it to $1/4$ resolution and add with a skip connection to the $1/4$ feature map from the backbone.

**Sensor depth to mesh point cloud feature transfer:** For 3D benchmarks like ScanNet [6] and ScanNet200 [41], the objective is to label a point cloud derived from a mesh rather than the depth map from the sensor. Hence, on those benchmarks, instead of upsampling the $1/8$ resolution feature map to $1/4$, we trilinearly interpolate features from the $1/8$ resolution feature map to the provided point cloud sampled from the mesh. This means: for each vertex in the mesh, we trilinearly interpolate from our computed 3D features to obtain interpolated features. We additionally similarly interpolate from the unprojected $1/4$ resolution feature map in the backbone, for an additive skip connection.

**Shared 2D-3D segmentation mask decoder:** Our segmentation decoder is a Transformer, similar to Mask2Former's decoder head, which takes as input upsampled 2D or 3D feature maps and outputs corresponding 2D or 3D segmentation masks and their semantic classes. Specifically, we instantiate a set of $N$ learnable object queries responsible for decoding individual instances. These queries are iteratively refined by a *Query Refinement* block, which consists of cross-attention to the upsampled features, followed by a self-attention between the queries. Except for the positional embeddings, all attention and query weights are shared between 2D and 3D. We use Fourier positional encodings in 2D, while in 3D we encode the XYZ coordinates of the 3D tokens with an MLP. The refined queries are used to predict instance masks and semantic classes. For mask prediction, the queries do a token-wise dot product with the highest-resolution upsampled features. For semantic class prediction, we use an MLP over the queries, mapping them to class logits. We refer readers to Mask2Former [4] for further details.

**Open vocabulary class decoder:** Drawing inspiration from prior open-vocabulary detection methods [19, 29, 61], we introduce an alternative classification head capable of handling an arbitrary number of semantic classes. This modification is essential for joint training on multiple datasets. Similar to BUTD-DETR [19] and GLIP [29], we supply the model with a *detection prompt* formed by concatenating object categories into a sentence (e.g., *"Chair. Table. Sofa."*) and encode it using RoBERTa [32]. In the query-refinement block, queries additionally attend to these text tokens before attending to the upsampled feature maps. For semantic class prediction, we first perform a dot-product operation between queries and language tokens, generating one logit per token in the detection prompt. The logits corresponding to prompt tokens for a specific object class are then averaged to derive per-class logits. This can handle multi-word noun phrases such as *"shower curtain"*, where we average the logits corresponding to *"shower"* and *"curtain"*. The segmentation masks are predicted by a pixel-/point-wise dot-product, in the same fashion as described earlier.

**Implementation details:** We initialize our model with pre-trained weights from Mask2Former [4] trained on COCO [31]. Subsequently, we train all parameters end-to-end, including both pre-trained and new parameters from 3D fusion layers. During training in 3D scenes, our model processes a sequence of $N$ consecutive frames, usually comprising 25 frames. At test time, we input all images in the scene to our model, with an average of 90 images per scene in ScanNet. We use vanilla closed-vocabulary decoding head for all experiments except when training jointly on 2D-3D datasets. There we use our open vocabulary class decoder that lets us handle different label spaces in these datasets. During training, we employ open vocabulary mask decoding for joint 2D and 3D datasets and vanilla closed-vocabulary decoding otherwise. Training continues until convergence on 2 NVIDIA A100s with 40 GB VRAM, with an effective batch size of 6 in 3D and 16 in 2D. For joint training on 2D and 3D datasets, we alternate sampling 2D and 3D batches with batch sizes of 3 and 8 per GPU, respectively. We adopt Mask2Former's strategy, using Hungarian matching for matching queries to ground truth instances and supervision losses. While our model is only trained for instance segmentation, it can perform semantic segmentation for free at test time like Mask2Former. We refer to Mask2Former [4] for more details.

# 4. Experiments

## 4.1. Evaluation on 3D benchmarks

**Datasets:** First, we test our model on 3D instance and semantic segmentation in the ScanNet [6] and ScanNet200 [41] benchmarks. The goal of these benchmarks is to label the point cloud extracted from the 3D mesh of a scene reconstructed from raw sensor data. ScanNet evaluates on 20 common semantic classes, while ScanNet200 uses 200 classes, which is more representative of the long-tailed object distribution encountered in the real world. We report results on the official validation split of these datasets here and on the official test split in the supplementary.

**Evaluation metrics:** We follow the standard evaluation metrics, namely mean Average Precision (mAP) for instance segmentation and mean Intersection over Union (mIoU) for semantic segmentation.

**Baselines:** In *instance segmentation*, our main baseline is the SOTA 3D method Mask3D [44]. For a thorough comparison, we train both Mask3D and our model with sensor RGB-D point cloud input and evaluate them on the benchmark-provided mesh-sampled point clouds. We also compare with the following recent and concurrent works: PBNet [58], QueryFormer [34] and MAFT [28]. QueryFormer and MAFT explore query initialization and refinement in a Mask3D-like architecture and thus have complementary advantages to ours. Unlike ODIN, these methods

Table 1. **Evaluation on 3D Benchmarks** ($^{\S}$ = trained by us using official codebase).

(a) **ScanNet Instance Segmentation Task.**

| | Model | mAP | mAP50 | mAP25 |
|---|---|---|---|---|
| Sensor RGBD Point Cloud | Mask3D$^{\S}$ [44] | 43.9 | 60.0 | 69.9 |
| | ODIN-ResNet50 (Ours) | 47.8 | 69.8 | **83.6** |
| | ODIN-Swin-B (Ours) | **50.0** | **71.0** | **83.6** |
| Mesh Sampled Point Cloud | SoftGroup [49] | 46.0 | 67.6 | 78.9 |
| | PBNet [58] | 54.3 | 70.5 | 78.9 |
| | Mask3D [44] | 55.2 | 73.7 | **83.5** |
| | QueryFormer [34] | 56.5 | 74.2 | 83.3 |
| | MAFT [28] | **58.4** | **75.9** | - |

(b) **ScanNet Semantic Segmentation Task.**

| | Model | mIoU |
|---|---|---|
| Sensor RGBD Point Cloud | MVPNet [20] | 68.3 |
| | BPNet [17] | 69.7 |
| | DeepViewAgg [40] | 71.0 |
| | ODIN-ResNet50 (Ours) | 73.3 |
| | ODIN-Swin-B (Ours) | **77.8** |
| Rendered RGBD Point Cloud | VMVF [26] | **76.4** |
| Mesh Sampled Point Cloud | Point Transformer v2 [51] | 75.4 |
| | Stratified Transformer [27] | 74.3 |
| | OctFormer [50] | 75.7 |
| | Swin3D-L [55] | **76.7** |
| Zero-Shot | OpenScene [37] | 54.2 |

(c) **ScanNet200 Instance Segmentation Task.**

| | Model | mAP | mAP50 | mAP25 |
|---|---|---|---|---|
| Sensor RGBD Point Cloud | Mask3D [44] $^{\S}$ | 15.5 | 21.4 | 24.3 |
| | ODIN-ResNet50 (Ours) | 25.6 | 36.9 | 43.8 |
| | ODIN-Swin-B (Ours) | **31.5** | **45.3** | **53.1** |
| Mesh Sampled Point Cloud | Mask3D [44] | 27.4 | 37.0 | 42.3 |
| | QueryFormer [34] | 28.1 | 37.1 | **43.4** |
| | MAFT [28] | **29.2** | **38.2** | 43.3 |
| Zero-Shot | OpenMask3D [47] | 15.4 | 19.9 | 23.1 |

(d) **ScanNet200 Semantic Segmentation Task.**

| | Model | mIoU |
|---|---|---|
| Sensor RGBD Point Cloud | ODIN-ResNet50 (Ours) | 35.8 |
| | ODIN-Swin-B (Ours) | **40.5** |
| Mesh Sampled Point Cloud | LGround [41] | 28.9 |
| | CeCo [60] | 32.0 |
| | Octformer [50] | **32.6** |

directly process 3D point clouds and initialize their weights from scratch. As motivated before, utilizing RGB-D input directly has several advantages, including avoiding costly mesh building processes, achieving closer integration of 2D and 3D perception, and leveraging pre-trained features and abundant 2D data.

In *semantic segmentation*, we compare with MVPNet [20], BPNet [17] and state-of-the-art DeepViewAgg [40] which directly operate on sensor RGB or RGB-D images and point clouds. We also compare with VMVF [26] which operates over rendered RGB-D images from the provided mesh, with heuristics for camera view sampling to avoid occlusions, ensures balanced scene coverage, and employs a wider field-of-view, though we note their code is not publicly available. Similar to ODIN, all of these methods utilize 2D pre-trained backbones. We also compare with Point-Transformer v2 [51], Stratified Transformer [27], OctFormer [50] and Swin3D-L [55] which process the mesh-sampled point cloud directly, without using any 2D pre-training. On the ScanNet200 semantic segmentation benchmark, we compare with SOTA OctFormer [50] and with CeCo [60], a method specially designed to fight class-imbalance in ScanNet200. These methods directly process the point cloud and do not use 2D image pre-trained weights. We also compare with LGround [41] which uses 2D CLIP pre-training. We also compare with zero-shot 2D foundation model-based 3D models of OpenScene [37] and OpenMask3D [47]. This comparison is unfair since they are not supervised within-domain, but we include them for completeness. The results are presented in Tab. 1. We draw the following conclusions:

**Performance drops with sensor point cloud as input** (Tab. 1a): Mask3D's performance drops from 55.2% mAP with mesh point cloud input to 43.9% mAP with sensor point cloud input. This is consistent with prior works [26, 40] in 3D semantic segmentation on ScanNet, which attributes the drop to misalignments caused by noise in camera poses, depth variations and post-processing steps.

**ODIN outperforms SOTA 3D methods with sensor point cloud input and underperforms them when baselines use mesh-sampled point clouds** (Tab. 1a): Our model significantly outperforms SOTA Mask3D model with sensor point cloud input and achieves comparable performance to methods using mesh-sampled point cloud input on the mAP25 metric while far behind on mAP metric, due to misalignments between the 3D mesh and the sensor point cloud.

**ODIN sets a new SOTA in semantic segmentation on ScanNet** (Tab. 1b) outperforming all methods on all setups

Table 2. **AI2THOR Semantic and Instance Segmentation.**

| Model | mAP | mAP50 | mAP25 | mIoU |
|---|---|---|---|---|
| Mask3D [44] | 60.6 | 70.8 | 76.6 | - |
| ODIN-ResNet50 (Ours) | 63.8 | **73.8** | **80.2** | **71.5** |
| ODIN-Swin-B (Ours) | **64.3** | 73.7 | 78.6 | 71.4 |

Table 3. **Embodied Instruction Following.** SR = success rate. GC = goal condition success rate.

| | TEACh | | | | ALFRED | | | |
|---|---|---|---|---|---|---|---|---|
| | Unseen | | Seen | | Unseen | | Seen | |
| | SR | GC | SR | GC | SR | GC | SR | GC |
| FILM [35] | - | - | - | - | 30.7 | 42.9 | 26.6 | 38.2 |
| HELPER [42] | 15.8 | 14.5 | 11.6 | 19.4 | 37.4 | 55.0 | 26.8 | 41.2 |
| HELPER + ODIN (OURS) | **18.6** | **18.6** | **13.8** | **26.6** | **47.7** | **61.6** | **33.5** | **47.1** |

including the models trained on the sensor, rendered and mesh sampled point clouds.

**ODIN sets a new instance segmentation SOTA on the long-tailed ScanNet200 dataset** (Tab. 1c) outperforming SOTA 3D models on all setups including the models trained on mesh-sampled point cloud, especially by a large margin in mAP25 metric, while exclusively utilizing sensor RGB-D data. This highlights the contribution of 2D features, particularly in detecting a long tail of class distribution where limited 3D data is available. We show more detailed results with performance on the head, common and tail classes in the appendix ( Sec. A.3).

**ODIN sets a new semantic segmentation SOTA on ScanNet200** (Tab. 1d), outperforming SOTA semantic segmentation models that use mesh point clouds.

### 4.2. Evaluation on multiview RGB-D in simulation

Using the AI2THOR [25] simulation environment with procedural homes from ProcThor [7], we collected RGB-D data for 1500 scenes (1200 training, 300 test) of similar size as ScanNet (more details in appendix, Sec. B). We train and evaluate our model and SOTA Mask3D [44] on the unprojected RGB-D images. As shown in Tab. 2, our model outperforms Mask3D by 3.7% mAP, showing strong performance in a directly comparable RGB-D setup. It suggests that current real-world benchmarks may restrain models that featurizes RGB-D sensor point clouds due to misalignments. We hope this encourages the community to also focus on directly collecting, labeling, and benchmarking RGB-D sensor data.

### 4.3. Embodied Instruction Following

We apply ODIN in the embodied setups of TEACh [36] and ALFRED [45] where agents have access to RGB, depth and camera poses and need to infer and execute task and action plans from dialogue segments and instructions, respectively. These agents operate in dynamic home environments and cannot afford expensive mesh building steps. Detecting objects well is critical for task success in both cases. Prior SOTA methods [36, 42] run per-view 2D instance segmentation models [4, 9] and link the detected instances using simple temporal reasoning regarding spatial and appearance proximity. Instead, ODIN processes its last $N$ egocentric views and segments objects instances directly in 3D. We equip HELPER [42], a state-of-the-art embodied model, with ODIN as its 3D object detection engine. We evaluate using Task Sucess Rate (SR) which checks if the entire task is executed successfully, and Goal Conditioned Success Rate (GC) which checks the proportion of satisfied subgoals across all episodes [36, 45]. We perform evaluation on "valid-seen" (houses similar to the training set) and "valid-unseen" (dissimilar houses) splits. In Tab. 3, we observe that HELPER with ODIN as its 3D object detector significantly outperforms HELPER that uses the original 2D detection plus linking perception pipeline.

Table 4. **Joint Training on Sensor RGB-D point cloud from ScanNet and 2D RGB images from COCO.**

| | ScanNet | | | COCO |
|---|---|---|---|---|
| | mAP | mAP50 | mAP25 | mAP |
| Mask3D [44] | 43.9 | 60.0 | 69.9 | ✗ |
| Mask2Former [4] | ✗ | ✗ | ✗ | **43.7** |
| ODIN (trained in 2D) | ✗ | ✗ | ✗ | 43.6 |
| ODIN (trained in 3D) | 47.8 | 69.8 | **83.6** | ✗ |
| ODIN (trained jointly) | **49.1** | **70.1** | 83.1 | 41.2 |

### 4.4. Ablations and Variants

We conduct our ablation experiments on the ScanNet dataset in Tab. 4 and Tab. 5. Our conclusions are:

**Joint 2D-3D training helps 3D perception** We compare joint training of ODIN on sensor RGB-D point clouds from ScanNet and 2D RGB images from COCO to variants trained independently on 2D and 3D data, all initialized from pre-trained COCO weights. Since there are different classes in ScanNet and COCO, we use our open-vocabulary semantic class-decoding head instead of the vanilla closed-vocabulary head. Results in Tab. 4 show that joint training yields a 1.3% absolute improvement in 3D, and causes a similar drop in 2D. This experiment indicates that a single architecture can perform well on both 2D and 3D tasks, thus indicating that we may not need to design vastly different architectures in either domain. However, the drop in 2D performance indicates a potential for further improvements in the architecture design to retain the performance in the 2D domain. Nevertheless, this experiment highlights the benefits of joint training with 2D datasets for 3D seg-

Table 5. **Ablations on ScanNet Dataset.**

(a) **Cross-View Contextualization.**

| Model | mAP | mIoU |
|---|---|---|
| ODIN (Ours) | **47.8** | 73.3 |
| No 3D Fusion | 39.3 | 73.2 |
| No interleaving | 41.7 | **73.6** |

(b) **Effect of Pre-Trained Features.**

| Model | mAP | mIoU |
|---|---|---|
| ODIN (Ours) | **47.8** | **73.3** |
| Only pre-trained backbone | 42.3 | 72.9 |
| No pre-trained features | 41.5 | 68.6 |

(c) **Effect of Freezing Backbone.**

| Model | ResNet50 | | Swin-B | |
|---|---|---|---|---|
| | mAP | mIoU | mAP | mIoU |
| ODIN (Ours) | **47.8** | 73.3 | **50.0** | **77.8** |
| With frozen backbone | 46.7 | **74.3** | 46.2 | 75.9 |

mentation in ODIN. Note that we do not jointly train on 2D and 3D datasets in any of our other experiments due to computational constraints.

**Cross-View fusion is crucial for instance segmentation but not for semantic segmentation** (Tab. 5a): removing 3D cross-view fusion layers results in an 8.5% mAP drop for instance segmentation, without any significant effect in semantic segmentation. Popular 2D-based 3D open vocabulary works [21, 37] without strong cross-view fusion only focus on semantic segmentation and thus could not uncover this issue. Row-3 shows a 6.1% mAP drop when cross-view 3D fusion happens after all within-view 2D layers instead of interleaving the within-view and cross-view fusion.

**2D pre-trained weight initialization helps** (Tab. 5b): initializing only the image backbone with pre-trained weights, instead of all layers (except the 3D fusion layers), results in a 5.5% mAP drop (row-2). Starting the entire model from scratch leads to a larger drop of 6.3% mAP (row-3). This underscores the importance of sharing as many parameters as possible with the 2D models to leverage the maximum possible 2D pre-trained weights.

**Stronger 2D backbones helps** (Tab. 5c): using Swin-B over ResNet50 leads to significant performance gains, suggesting that ODIN can directly benefit from advancements in 2D computer vision.

**Finetuning everything including the pre-trained parameters helps** (Tab. 5c): ResNet50's and Swin's performance increases substantially when we fine-tune all parameters. Intuitively, unfreezing the backbone allows 2D layers to adapt to cross-view fused features better. Thus, we keep our backbone unfrozen in all experiments.

**Supplying 2D features directly to 3D models does not help:** Concatenating 2D features with XYZ+RGB as input to Mask3D yields 53.8% mAP performance, comparable to 53.3% of the baseline model with only XYZ+RGB as input.

### 4.5. Additional Experiments

We show evaluations on the hidden test set of ScanNet and ScanNet200 in Sec. A.1, results and comparisons with baselines on S3DIS [1] and MatterPort3D [2] datasets in Sec. A.2 and performance gains in 2D perception with increasing context views in Sec. A.4.

### 4.6. Limitations

Our experiments reveal the following limitations for ODIN: Firstly, like other top-performing 3D models, it depends on accurate depth and camera poses. Inaccurate depth or camera poses cause a sharp decrease in performance (similar to other 3D models, like Mask3D). In our future work, we aim to explore unifying depth and camera pose estimation with semantic scene parsing, thus making 3D models more resilient to noise. Secondly, in this paper, we limited our scope to exploring the design of a unified architecture without scaling up 3D learning by training on diverse 2D and 3D datasets jointly. We aim to explore this in future to achieve strong generalization to in-the-wild scenarios, akin to the current foundational 2D perception systems. Our results suggest a competition between 2D and 3D segmentation performance when training ODIN jointly on both modalities. Exploring ways to make 2D and 3D training more synergistic is a promising avenue for future work.

## 5. Conclusion

We presented ODIN, a model for 2D and 3D instance segmentation that can parse 2D images and 3D point clouds alike. ODIN represents both 2D images and 3D feature clouds as a set of tokens that differ in their positional encodings which represent 2D pixel coordinates for 2D tokens and 3D XYZ coordinates for 3D tokens. Our model alternates between within-image featurization and cross-view featurization. It achieves SOTA performance in ScanNet200 and AI2THOR instance segmentation benchmarks, outperforms all methods operating on sensor point clouds and achieves competent performance to methods operating over mesh-sampled pointcloud. Our experiments show that ODIN outperforms alternative models that simply augment 3D point cloud models with 2D image features as well as ablative versions of our model that do not alternate between 2D and 3D information fusion, do not co-train across 2D and 3D and do no pre-train the 2D backbone.

[†] We do not use the expensive DB-SCAN post-processing of Mask3D, and hence it gets 53.3% mAP instead of 55.2% as reported by their paper

## 6. Acknowledgements

The authors express gratitude to Wen-Hsuan Chu, Mihir Prabhudesai, and Alexander Swerdlow for their valuable feedback on the early draft of this work. Special thanks to Tsung-Wei Ke for insightful discussions throughout the project. We thank the Microsoft Turing Team for providing us with GPU resources during the initial development phase of this project. This work is supported by ONR award N00014-23-1-2415, Sony AI, DARPA Machine Common Sense, an Amazon faculty award, and an NSF CAREER award.

## Appendix A. Experiments

### A.1. Evaluations on ScanNet and ScanNet200 Hidden Test Sets

We submit ODIN to official test benchmarks of ScanNet [6] and ScanNet200 [41]. Following prior works, we train ODIN on a combination of train and validation scenes. Unlike some approaches that employ additional tricks like DBSCAN [44], ensembling models [27], additional specialized augmentations [51], additional pre-training on other datasets [55], finer grid sizes [50] and multiple forward passes through points belonging to the same voxel, our method avoid any such bells and whistles.

The results are shown in Tab. 6. All conclusions from results on the validation set of these datasets as discussed in the main paper are applicable here. On the ScanNet benchmark, ODIN achieves close to SOTA performance on semantic segmentation and mAP25 metric of Instance Segmentation while being significantly worse on mAP metric due to misalignments between sensor and mesh sampled point clouds. On ScanNet200 benchmark, ODIN sets a new SOTA on semantic segmentation and mAP50/mAP25 metric of Instance Segmentation, while achieving close to SOTA performance on mAP metric. Notably ODIN is the first method that operates over sensor RGB-D data for instance segmentation and achieves competitive performance to models operating over mesh-sampled point clouds.

### A.2. Evaluation on S3DIS and Matterport3D

We also benchmark ODIN on Matterport3D [2] and S3DIS [1] datasets.

**Matterport:** Matterport3D comprises 90 building-scale scenes, further divided into individual rooms, with 1554 training rooms and 234 validation rooms. The dataset provides a mapping from each room to the camera IDs that captured images for that room. After discarding 158 training rooms and 18 validation rooms without a valid camera mapping, we are left with 1396 training rooms and 158 validation rooms. For instance segmentation results, we train the state-of-the-art Mask3D [44] model on the same data (reduced set after discarding invalid rooms). For semantic segmentation, we conduct training and testing on the reduced set, while baseline numbers are taken from the OpenScene [37] paper, trained and tested on the original data. Given the small size of the discarded data, we do not anticipate significant performance differences. The official benchmark of Matterport3D tests on 21 classes; however, OpenScene also evaluates on 160 classes to compare with state-of-the-art models on long-tail distributions. We follow them and report results in both settings.

**S3DIS:** S3DIS comprises 6 building-scale scenes, typically divided into 5 for training and 1 for testing. The dataset provides raw RGB-D images, captured panorama images, and images rendered from the mesh obtained after reconstructing the original sensor data. Unlike Matterport3D, S3DIS do not provide undistorted raw images; thus, we use the provided rendered RGB-D images. Some rooms in S3DIS have major misalignments between RGB-D images and point clouds, which we partially address by incorporating fixes from DeepViewAgg [40] and introducing our own adjustments. Despite these fixes, certain scenes still exhibit significantly low overlap between RGB-D images and the provided mesh-sampled point cloud. To mitigate this, we query images from other rooms and verify their overlap with the provided point cloud for a room. This partially helps in addressing the low overlap issue.

The official S3DIS benchmark evaluates 13 classes. Due to the dataset's small size, some models pre-train on additional datasets like ScanNet, as seen in SoftGroup [49], and on Structured3D datasets [59], consisting of 21,835 rooms, as done by Swin3D-L [55]. Similar to Mask3D [44], we report results in both settings of training from scratch and starting from weights trained on ScanNet.

Like ScanNet and ScanNet200, both S3DIS and Matterport3D undergo post-processing of collected RGB-D data to construct a mesh, from which a point cloud is sampled and labeled. Hence, we train both Mask3D [44] and our model using RGB-D sensor point cloud data and evaluate on the benchmark-provided point cloud. Additionally, we explore model variants by training and testing them on the mesh-sampled point cloud for comparative analysis.

The results are shown in Tab. 7. We draw the following conclusions:

ODIN outperforms SOTA 3D models on Matterport3D Instance Segmentation Benchmark across all settings (Tab. 7a)

ODIN sets a new state-of-the-art on Matterport3D Semantic Segmentation Benchmark (Tab. 7b): Our model achieves superior performance in both the 21 and 160 class settings. It also largely outperforms OpenScene [37] on both settings. OpenScene is a zero-shot method while ODIN is supervised in-domain, making this comparison unfair. However, OpenScene notes that their zero-shot model outperforms fully-supervised models in 160 class setup as their model is robust

Table 6. **Evaluation on Test Set of Established 3D Benchmarks.**

(a) **Comparison on ScanNet for Instance Segmentation Task.**

| Input | Model | mAP | mAP50 | mAP25 |
|---|---|---|---|---|
| Sensor RGBD Point Cloud | ODIN-Swin-B (Ours) | **47.7** | **71.2** | **86.2** |
| Mesh Sampled Point Cloud | SoftGroup [49] | 50.4 | 76.1 | 86.5 |
| | PBNet [58] | 57.3 | 74.7 | 82.5 |
| | Mask3D [44] | 56.6 | 78.0 | 87.0 |
| | QueryFormer [34] | 58.3 | **78.7** | **87.4** |
| | MAFT [28] | **59.6** | 78.6 | 86.0 |

(b) **Comparison on ScanNet for Semantic Segmentation Task.**

| Input | Model | mIoU |
|---|---|---|
| Sensor RGBD Point Cloud | MVPNet [20] | 64.1 |
| | BPNet [17] | **74.9** |
| | DeepViewAgg [40] | - |
| | ODIN-Swin-B (Ours) | 74.4 |
| Rendered RGBD Point Cloud | VMVF [26] | **74.6** |
| Mesh Sampled Point Cloud | Point Transformer v2 [51] | 75.2 |
| | Stratified Transformer [27] | 74.7 |
| | OctFormer [50] | 76.6 |
| | Swin3D-L [55] | **77.9** |
| Zero-Shot | OpenScene [37] | - |

(c) **Comparison on ScanNet200 for Instance Segmentation Task.**

| | Model | mAP | mAP50 | mAP25 |
|---|---|---|---|---|
| Sensor RGBD Point Cloud | ODIN-Swin-B (Ours) | **27.2** | **39.4** | **47.5** |
| Mesh Sampled Point Cloud | Mask3D [44] | **27.8** | **38.8** | **44.5** |
| | QueryFormer [34] | - | - | - |
| | MAFT [28] | - | - | - |
| Zero-Shot | OpenMask3D [47] | - | - | - |

(d) **Comparison on ScanNet200 for Semantic Segmentation Task.**

| Input | Model | mIoU |
|---|---|---|
| Sensor RGBD Point Cloud | ODIN-Swin-B (Ours) | **36.8** |
| Mesh Sampled Point Cloud | LGround [41] | 27.2 |
| | CeCo [60] | **34.0** |
| | Octformer [50] | 32.6 |

Table 7. **Evaluation on Matterport3D [2] and S3DIS [1] datasets.**

(a) **Comparison on Matterport3D for Instance Segmentation Task.**

| | | 21 | | 160 | |
|---|---|---|---|---|---|
| Input | Model | mAP | mAP25 | mAP | mAP25 |
| Sensor RGBD Point Cloud | Mask3D [44] | 7.2 | 16.8 | 2.5 | 10.9 |
| | ODIN-ResNet50 (Ours) | 22.5 | 56.4 | 11.5 | 27.6 |
| | ODIN-Swin-B (Ours) | **24.7** | **63.8** | **14.5** | **36.8** |
| Mesh Sampled Point Cloud | Mask3D [44] | **22.9** | **55.9** | **11.3** | **23.9** |

(b) **Comparison on Matterport3D for Semantic Segmentation Task.**

| | | 21 | | 160 | |
|---|---|---|---|---|---|
| Input | Model | mIoU | mAcc | mIoU | mAcc |
| Sensor RGBD Point Cloud | ODIN-ResNet50 (Ours) | 54.5 | 65.8 | 22.4 | 28.5 |
| | ODIN-Swin-B (Ours) | **57.3** | **69.4** | **28.6** | **38.2** |
| Mesh Sampled Point Cloud | TextureNet [18] | - | 63.0 | - | - |
| | DCM-Net [43] | - | **67.2** | - | - |
| | MinkowskiNet [5] | **54.2** | 64.6 | - | **18.4** |
| Zero-Shot | OpenScene [37] | **42.6** | **59.2** | - | **23.1** |

(c) **Comparison on S3DIS Area5 for Instance Segmentation Task.** ($^\dagger$ = uses additional data)

| | Model | mAP | mAP50 | mAP25 |
|---|---|---|---|---|
| RGBD Point Cloud | Mask3D [44] | 40.7 | 54.6 | 64.2 |
| | Mask3D [44] $^\dagger$ | 41.3 | 55.9 | 66.1 |
| | ODIN-ResNet50 (Ours) | 36.3 | 48.0 | 61.2 |
| | ODIN-ResNet50 $^\dagger$ (Ours) | **44.7** | **57.7** | 67.5 |
| | ODIN-Swin-B $^\dagger$ (Ours) | 43.0 | 56.4 | **70.0** |
| Mesh Sampled Point Cloud | SoftGroup [49] $^\dagger$ | 51.6 | 66.1 | - |
| | Mask3D [44] | 56.6 | 68.4 | 75.2 |
| | Mask3D [44] $^\dagger$ | **57.8** | **71.9** | **77.2** |
| | QueryFormer [34] | 57.7 | 69.9 | - |
| | MAFT [28] | - | 69.1 | 75.7 |

(d) **Comparison on S3DIS for Semantic Segmentation Task.** ($^\dagger$ = uses additional data)

| Input | Model | mIoU |
|---|---|---|
| RGBD Point Cloud | MVPNet [20] | 62.4 |
| | VMVF [26] | 65.4 |
| | DeepViewAgg [40] | 67.2 |
| | ODIN-ResNet50 (Ours) | 59.7 |
| | ODIN-ResNet50 $^\dagger$ (Ours) | 66.8 |
| | ODIN-Swin-B $^\dagger$ (Ours) | **68.6** |
| Mesh Sampled Point Cloud | Point Transformer v2 [51] | 71.6 |
| | Stratified Transformer [27] | 72.0 |
| | Swin3D-L [55] $^\dagger$ | **74.5** |

to rare classes while the supervised models can severely suffer in segmenting long-tail. ConceptFusion [21], another open-vocabulary 3D segmentation model, also draws a similar conclusion. With this result, we point to a possibility of supervising in 3D while also being robust to long-tail by simply utilizing the strong 2D pre-trained weight initialization.

On S3DIS Instance Segmentation Benchmark (Tab. 7c), in the setup where baseline Mask3D start from ScanNet pre-trained checkpoint, our model outperforms them in the RGBD point cloud setup but obtains lower performance compared to mesh sampled point cloud methods and when compared on the setup where all models train from scratch.

On S3DIS Semantic Segmentation Benchmark (Tab. 7d, ODIN trained with ScanNet weight initialization outperforms all RGBD point cloud based methods, while achieving competitive performance on mesh sampled point cloud. When trained from scratch, it is much worse than other baselines. Given the limited dataset size of S3DIS with only 200 training scenes, we observe severe overfitting.

### A.3. ScanNet200 Detailed Results

ScanNet200 [41] categorizes its 200 object classes into three groups—*Head*, *Common*, and *Tail*—each comprising 66, 68, and 66 categories, respectively. In Tab. 8, we provide a detailed breakdown of the ScanNet200 results across these splits. We observe that in comparison to SOTA Mask3D model trained on mesh-sampled point cloud, ODIN achieves lower performance on *Head* classes, while significantly better performance on *Common* and *Tail* classes. This highlights the contribution of effectively utilizing 2D pre-trained features, particularly in detecting a long tail of class distribution where limited 3D data is available.

### A.4. Variation of Performance with Number of Views

We examine the influence of the number of views on segmentation performance using the AI2THOR dataset, specifically focusing on the 2D mAP performance metric. The evaluation is conducted by varying the number of *context* images surrounding a given *query* RGB image. Starting from a single-view without any context (N=0), we increment N to 5, 10, 20, 40, 60, and finally consider all images in the scene as context. ODIN takes these $N + 1$ RGB-D images as input, predicts per-pixel instance segmentation for each image, and assesses the 2D mAP performance on the *query* image. The results, depicted in Fig. 3, show a continuous increase in 2D mAP with the growing number of views. This observation underscores the advantage of utilizing multiview RGB-D images over single-view RGB images whenever feasible.

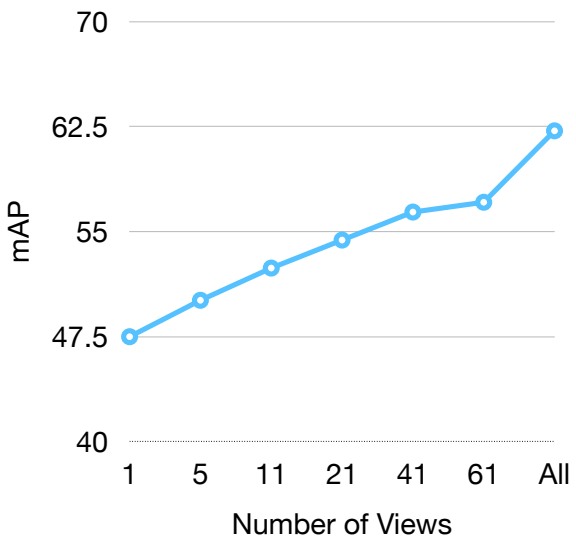


Figure 3. **2D mAP Performance Variation with increasing number of context views used**

### A.5. Inference Time

We assess the inference time of Mask3D and ODIN by averaging the forward pass time of each model across the entire validation set, utilizing a 40 GB VRAM A100. When fed the mesh-sampled point cloud directly, Mask3D achieves an inference time of 228ms. When provided with the sensor point cloud as input, the inference time increases to 864 ms. Mask3D with sensor point cloud is slower than with mesh point cloud because at the same voxel size (0.02m), more voxels are occupied in sensor point cloud ( 110k on avg.) compared to mesh point clouds ( 64k on avg.) as mesh-cleaning sometimes discards large portion of the scene. The transfer of features from the sensor point cloud to the mesh point cloud adds an extra 7 ms. ODIN-SwinB, which operates over the sensor point cloud, has an inference time of 960ms.

## Appendix B. Additional Implementation Details

The detailed components of our architecture and their descriptions are presented in Fig. 4.

More implementation details are presented below:

**Augmentations:** For RGB image augmentation, we implement the Large Scale Jittering Augmentation method from Mask2Former [4], resizing images to a scale between 0.1 and 2.0. We adjust intrinsics accordingly post-augmentation and apply color jittering to RGB images. Training involves a consecutive set of $N$ images, typically set to 25. With a 50% probability, we randomly sample $k$

Table 8. **Detailed ScanNet200 results for Instance Segmentation** ($^{\S}$ = trained by us using official codebase)

| Input | Model | All | | | Head | | | Common | | | Tail | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | mAP | mAP50 | mAP25 | mAP | mAP50 | mAP25 | mAP | mAP50 | mAP25 | mAP | mAP50 | mAP25 |
| Sensor RGBD point cloud | Mask3D $^{\S}$ [44] | 15.5 | 21.4 | 24.3 | 21.9 | 31.4 | 37.1 | 13.0 | 17.2 | 18.9 | 7.9 | 10.3 | 11.5 |
| | ODIN-ResNet50 (Ours) | 25.6 | 36.9 | 43.8 | 34.8 | 51.1 | 63.9 | 23.4 | 33.4 | 37.9 | 17.8 | 24.9 | 28.1 |
| | ODIN-Swin-B (Ours) | **31.5** | **45.3** | **53.1** | 37.5 | 54.2 | **66.1** | **31.6** | **43.9** | **50.2** | **24.1** | **36.6** | **41.2** |
| Mesh Sampled point cloud | Mask3D [44] | 27.4 | 37.0 | 42.3 | **40.3** | **55.0** | 62.2 | 22.4 | 30.6 | 35.4 | 18.2 | 23.2 | 27.0 |

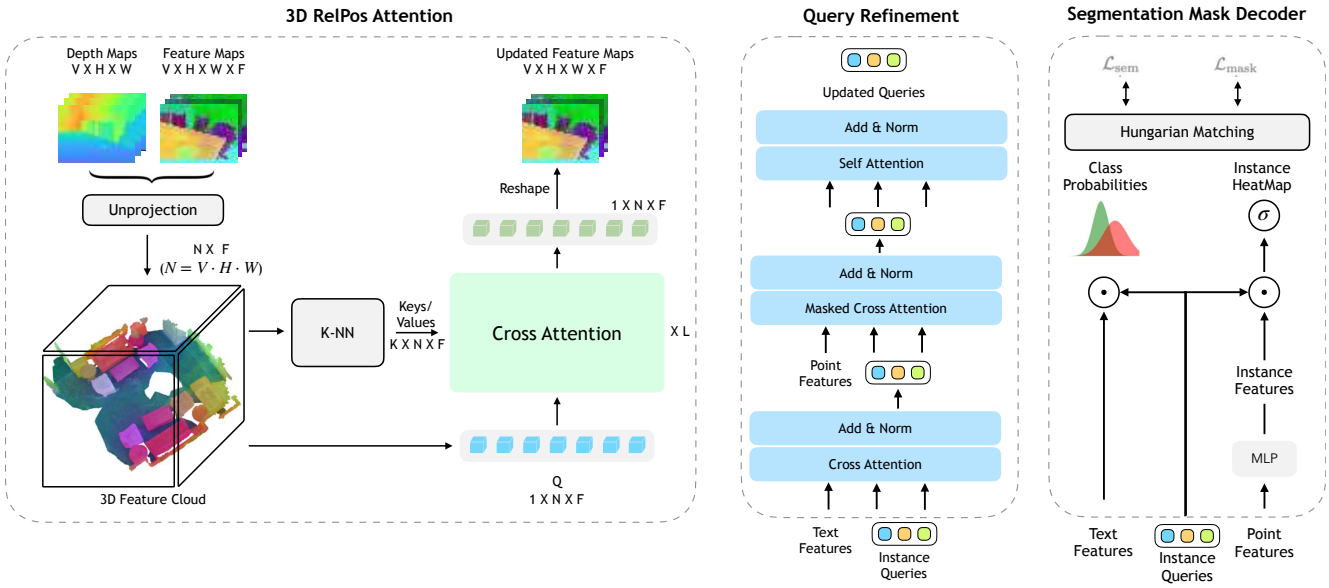


Figure 4. **Detailed ODIN Architecture Components**: On the **Left** is the 3D RelPos Attention module which takes as input the depth, camera parameters and feature maps from all views, lifts the features to 3D to get 3D tokens. Each 3D token serves as a query. The K-Nearest Neighbors of each 3D token become the corresponding keys and values. The 3D tokens attend to their neighbours for L layers and update themselves. Finally, the 3D tokens are mapped back to the 2D feature map by simply reshaping the 3D feature cloud to 2D multi-view feature maps. On the **Middle** is the query refinement block where queries first attend to the text tokens, then to the visual tokens and finally undergo self-attention. The text features are optional and are only used in the open-vocabulary decoder setup. On the **Right** is the segmentation mask decoder head where the queries simply perform a dot-product with visual tokens to decode the segmentation heatmap, which can be thresholded to obtain the segmentation mask. In the Open-Vocabulary decoding setup, the queries also perform a dot-product with text tokens to decode a distribution over individual words. In a closed vocabulary decoding setup, queries simply pass through an MLP to predict a distribution over classes.

images from the range $[1, N]$ instead of using all $N$ images. Additionally, instead of consistently sampling $N$ consecutive images, we randomly skip $k$ images in between, where $k$ ranges from 1 to 4.

For 3D augmentations, we adopt the Mask3D [44] approach, applying random 3D rotation, scaling, and jitter noise to the unprojected XYZs. Elastic distortion and random flipping augmentations from Mask3D are omitted due to a slight drop in performance observed in our initial experiments.

**Image Resolutions** We use a resolution of $256 \times 256$ for ScanNet, $512 \times 512$ for ScanNet200, and AI2THOR. In our AI2THOR experiments, we discovered that employing higher image resolutions enhances the detection of smaller objects, with no noticeable impact on the detection of larger ScanNet-like objects. This observation was confirmed in ScanNet, where we experimented with $512 \times 512$ image resolutions and did not observe any discernible benefit.

**Interpolation** Throughout our model, interpolations are employed in various instances, such as when upsampling the feature map from $1/8$th resolution to $1/4$th. In cases involving depth, we unproject feature maps to 3D and perform trilinear interpolation, as opposed to directly applying bilinear interpolation on the 2D feature maps. For upsampling/downsampling the depth maps, we use the nearest interpolation. Trilinear interpolation proves crucial for obtaining accurate feature maps, particularly at 2D object boundaries like table and floor edges. This is because nearest depth interpolation may capture depth from either the table or the floor. Utilizing trilinear upsampling of feature

maps ensures that if the upsampled depth is derived from the floor, it interpolates features from floor points rather than table points.

**Use of Segments:** Some datasets, such as ScanNet and ScanNet200, provide supervoxelization of the point cloud, commonly referred to as *segments*. Rather than directly segmenting all input points, many 3D methods predict outputs over these segments. Specifically, Mask3D [44] featurizes the input points and then conducts mean pooling over the features of points belonging to a segment, resulting in one feature per segment. Following prior work, we also leverage segments in a similar manner. We observe that utilizing segments is crucial for achieving good mAP performance, while it has no discernible impact on mAP25 performance. We suspect that this phenomenon may arise from the annotation process of these datasets. Humans were tasked with labelling segments rather than individual points, ensuring that all points within a segment share the same label. Utilizing segments with our models guarantees that the entire segment is labelled with the same class. It's worth noting that in AI2THOR, our method and the baselines do not utilize these segments, as they are not available.

**Post-hoc output transfer vs feature transfer:** ODIN takes the sensor point cloud as input and generates segmentation output on the benchmark-provided point cloud. In this process, we featurize the sensor point cloud and transfer these features from the sensor point cloud to the benchmark-provided point cloud. Subsequently, we predict segmentation outputs on this benchmark-provided feature cloud and supervise the model with the labels provided in the dataset. An alternative approach involves segmenting and supervising the sensor RGB-D point cloud and later transferring the segmentation output to the benchmark point cloud for evaluation. We experimented with both strategies and found them to yield similar results. However, as many datasets provide segmentation outputs only on the point cloud, transferring labels to RGB-D images for the latter strategy requires careful consideration. This is due to the sparser nature of the provided point cloud compared to the RGB-D sensor point cloud, and factors such as depth noise and misalignments can contribute to low-quality label transfer. Consequently, we opt for the former strategy in all our experiments.

**Depth Hole-Infilling:** The sensor-collected depth maps usually have holes around object boundaries and shiny/transparent surfaces. We perform simple OpenCV depth inpainting to fill these holes. We tried using neural-based depth completion methods and NERF depth-inpainting but did not observe significant benefits.

**AI2THOR Data Collection:** AI2THOR [25] is an embodied simulator where an agent can navigate within a house, execute actions, and capture RGB-D images of the scene. We load the structurally generated houses from ProcTHOR [7] into the AI2THOR simulator, and place an agent randomly at a navigable point provided by the simulator. The agent performs a single random rotation around its initial location and captures an RGB-D frame. This process is repeated, with the agent spawning at another random location, until either all navigable points are exhausted or a maximum of $N = 120$ frames is collected. While ProcTHOR offers 10,000 scenes, we randomly select only 1,500 scenes to match the size of ScanNet. Additionally, we retain scenes with fewer than 100 objects, as our model utilizes a maximum of 100 object queries.

## Appendix C. Qualitative Results

Fig. 5 shows qualitative visualizations of ODIN for various 3D and 2D datasets.

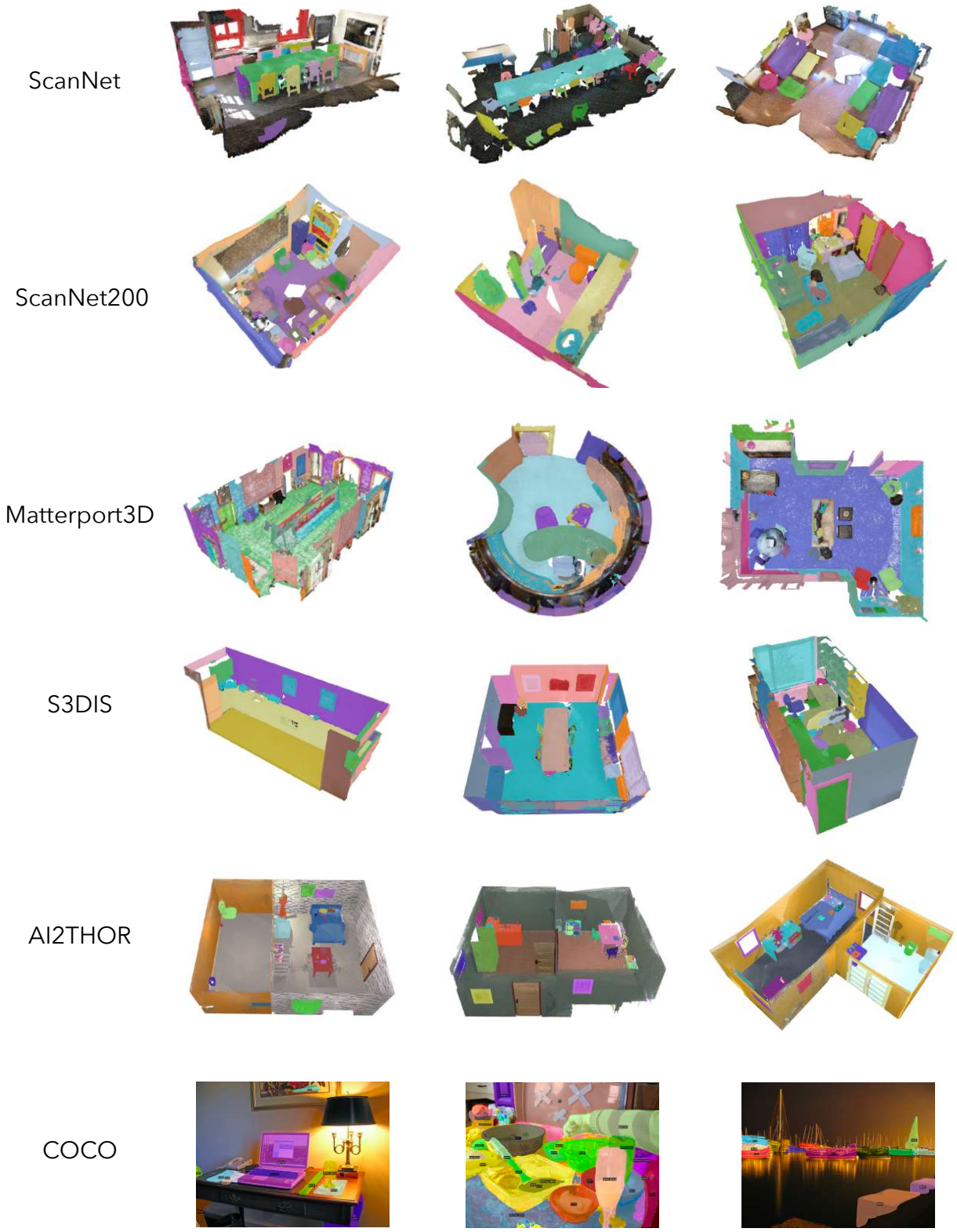


Figure 5. Qualitative Results on various 3D and 2D datasets

## References


[1] Iro Armeni, Sasha Sax, Amir R Zamir, and Silvio Savarese. Joint 2d-3d-semantic data for indoor scene understanding. *arXiv preprint arXiv:1702.01105*, 2017. 2, 8, 9, 10

[2] Angel Chang, Angela Dai, Thomas Funkhouser, Maciej Halber, Matthias Niessner, Manolis Savva, Shuran Song, Andy Zeng, and Yinda Zhang. Matterport3d: Learning from rgb-d data in indoor environments. *arXiv preprint arXiv:1709.06158*, 2017. 2, 8, 9, 10

[3] Shaoyu Chen, Jiemin Fang, Qian Zhang, Wenyu Liu, and Xinggang Wang. Hierarchical aggregation for 3d instance segmentation. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15447–15456, 2021. 2

[4] Bowen Cheng, Ishan Misra, Alexander G. Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. 2022. 1, 2, 3, 4, 5, 7, 11

[5] Christopher Choy, JunYoung Gwak, and Silvio Savarese. 4d spatio-temporal convnets: Minkowski convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3075–3084, 2019. 2, 10

[6] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5828–5839, 2017. 1, 2, 3, 5, 9

[7] Matt Deitke, Eli VanderBilt, Alvaro Herrasti, Luca Weihs, Kiana Ehsani, Jordi Salvador, Winson Han, Eric Kolve, Aniruddha Kembhavi, and Roozbeh Mottaghi. Procthor: Large-scale embodied ai using procedural generation. *Advances in Neural Information Processing Systems*, 35:5982–5994, 2022. 2, 7, 13

[8] Runyu Ding, Jihan Yang, Chuhui Xue, Wenqing Zhang, Song Bai, and Xiaojuan Qi. Pla: Language-driven open-vocabulary 3d scene understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7010–7019, 2023. 1

[9] Bin Dong, Fangao Zeng, Tiancai Wang, Xiangyu Zhang, and Yichen Wei. Solq: Segmenting objects by learning queries. *Advances in Neural Information Processing Systems*, 34: 21898–21909, 2021. 7

[10] Shichao Dong, Fayao Liu, and Guosheng Lin. Leveraging large-scale pretrained vision foundation models for label-efficient 3d point cloud segmentation. *arXiv preprint arXiv:2311.01989*, 2023. 3

[11] Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Graph neural networks with learnable structural and positional representations. *arXiv preprint arXiv:2110.07875*, 2021. 4

[12] Kyle Genova, Xiaoqi Yin, Abhijit Kundu, Caroline Pantofaru, Forrester Cole, Avneesh Sud, Brian Brewington, Brian Shucker, and Thomas Funkhouser. Learning 3d semantic segmentation with only 2d image supervision. In *2021 International Conference on 3D Vision (3DV)*, pages 361–372. IEEE, 2021. 3

[13] Rohit Girdhar, Mannat Singh, Nikhila Ravi, Laurens van der Maaten, Armand Joulin, and Ishan Misra. Omnivore: A single model for many visual modalities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16102–16112, 2022. 3

[14] Huy Ha and Shuran Song. Semantic abstraction: Open-world 3d scene understanding from 2d vision-language models. In *6th Annual Conference on Robot Learning*, 2022. 1, 3

[15] Lei Han, Tian Zheng, Lan Xu, and Lu Fang. Occuseg: Occupancy-aware 3d instance segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2940–2949, 2020. 2

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 2, 3

[17] Wenbo Hu, Hengshuang Zhao, Li Jiang, Jiaya Jia, and Tien-Tsin Wong. Bidirectional projection network for cross dimension scene understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14373–14382, 2021. 3, 6, 10

[18] Jingwei Huang, Haotian Zhang, Li Yi, Thomas Funkhouser, Matthias Nießner, and Leonidas J Guibas. Texturenet: Consistent local parametrizations for learning from high-resolution signals on meshes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4440–4449, 2019. 10

[19] Ayush Jain, Nikolaos Gkanatsios, Ishita Mediratta, and Katerina Fragkiadaki. Bottom up top down detection transformers for language grounding in images and point clouds. In *European Conference on Computer Vision*, pages 417–433. Springer, 2022. 5

[20] Maximilian Jaritz, Jiayuan Gu, and Hao Su. Multi-view pointnet for 3d scene understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pages 0–0, 2019. 3, 6, 10

[21] Krishna Murthy Jatavallabhula, Alihusein Kuwajerwala, Qiao Gu, Mohd Omama, Tao Chen, Shuang Li, Ganesh Iyer, Soroush Saryazdi, Nikhil Keetha, Ayush Tewari, et al. Conceptfusion: Open-set multimodal 3d mapping. *arXiv preprint arXiv:2302.07241*, 2023. 1, 3, 8, 11

[22] Li Jiang, Hengshuang Zhao, Shaoshuai Shi, Shu Liu, Chi-Wing Fu, and Jiaya Jia. Pointgroup: Dual-set point grouping for 3d instance segmentation. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4866–4875, 2020. 2

[23] Justin Kerr, Chung Min Kim, Ken Goldberg, Angjoo Kanazawa, and Matthew Tancik. Lerf: Language embedded radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 19729–19739, 2023. 1, 3

[24] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. *arXiv preprint arXiv:2304.02643*, 2023. 3

[25] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani,

Daniel Gordon, Yuke Zhu, et al. Ai2-thor: An interactive 3d environment for visual ai. *arXiv preprint arXiv:1712.05474*, 2017. 2, 7, 13

[26] Abhijit Kundu, Xiaoqi Yin, Alireza Fathi, David Ross, Brian Brewington, Thomas Funkhouser, and Caroline Pantofaru. Virtual multi-view fusion for 3d semantic segmentation. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXIV 16*, pages 518–535. Springer, 2020. 3, 6, 10

[27] Xin Lai, Jianhui Liu, Li Jiang, Liwei Wang, Hengshuang Zhao, Shu Liu, Xiaojuan Qi, and Jiaya Jia. Stratified transformer for 3d point cloud segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8500–8509, 2022. 6, 9, 10

[28] Xin Lai, Yuhui Yuan, Ruihang Chu, Yukang Chen, Han Hu, and Jiaya Jia. Mask-attention-free transformer for 3d instance segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3693–3703, 2023. 1, 2, 5, 6, 10

[29] Liunian Harold Li, Pengchuan Zhang, Haotian Zhang, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, et al. Grounded language-image pre-training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10965–10975, 2022. 5

[30] Zhihao Liang, Zhihao Li, Songcen Xu, Mingkui Tan, and Kui Jia. Instance segmentation in 3d scenes using semantic superpoint tree networks. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2763–2772, 2021. 2

[31] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014. 5

[32] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019. 5

[33] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021. 2, 3

[34] Jiahao Lu, Jiacheng Deng, Chuxin Wang, Jianfeng He, and Tianzhu Zhang. Query refinement transformer for 3d instance segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18516–18526, 2023. 2, 5, 6, 10

[35] So Yeon Min, Devendra Singh Chaplot, Pradeep Kumar Ravikumar, Yonatan Bisk, and Ruslan Salakhutdinov. Film: Following instructions in language with modular methods. In *International Conference on Learning Representations*, 2021. 7

[36] Aishwarya Padmakumar, Jesse Thomason, Ayush Shrivastava, Patrick Lange, Anjali Narayan-Chen, Spandana Gella, Robinson Piramuthu, Gokhan Tur, and Dilek Hakkani-Tur. Teach: Task-driven embodied agents that chat. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2017–2025, 2022. 2, 7

[37] Songyou Peng, Kyle Genova, Chiyu Jiang, Andrea Tagliasacchi, Marc Pollefeys, Thomas Funkhouser, et al. Openscene: 3d scene understanding with open vocabularies. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 815–824, 2023. 1, 3, 6, 8, 9, 10

[38] Guocheng Qian, Xingdi Zhang, Abdullah Hamdi, and Bernard Ghanem. Pix4point: Image pretrained transformers for 3d point cloud understanding. 2022. 3

[39] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 3

[40] Damien Robert, Bruno Vallet, and Loic Landrieu. Learning multi-view aggregation in the wild for large-scale 3d semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5575–5584, 2022. 1, 3, 6, 9, 10

[41] David Rozenberszki, Or Litany, and Angela Dai. Language-grounded indoor 3d semantic segmentation in the wild. In *European Conference on Computer Vision*, pages 125–141. Springer, 2022. 1, 2, 5, 6, 9, 10, 11

[42] Gabriel Sarch, Yue Wu, Michael J Tarr, and Katerina Fragkiadaki. Open-ended instructable embodied agents with memory-augmented large language models. *arXiv preprint arXiv:2310.15127*, 2023. 2, 7

[43] Jonas Schult, Francis Engelmann, Theodora Kontogianni, and Bastian Leibe. Dualconvmesh-net: Joint geodesic and euclidean convolutions on 3d meshes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8612–8622, 2020. 10

[44] Jonas Schult, Francis Engelmann, Alexander Hermans, Or Litany, Siyu Tang, and Bastian Leibe. Mask3d: Mask transformer for 3d semantic instance segmentation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8216–8223. IEEE, 2023. 1, 2, 5, 6, 7, 9, 10, 12, 13

[45] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10740–10749, 2020. 7

[46] Yawar Siddiqui, Lorenzo Porzi, Samuel Rota Bulò, Norman Müller, Matthias Nießner, Angela Dai, and Peter Kontschieder. Panoptic lifting for 3d scene understanding with neural fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9043–9052, 2023. 1, 3

[47] Ayça Takmaz, Elisabetta Fedele, Robert W Sumner, Marc Pollefeys, Federico Tombari, and Francis Engelmann. Open-

mask3d: Open-vocabulary 3d instance segmentation. *arXiv preprint arXiv:2306.13631*, 2023. 1, 3, 6, 10

[48] Nikolaos Tsagkas, Oisin Mac Aodha, and Chris Xiaoxuan Lu. Vl-fields: Towards language-grounded neural implicit spatial representations. *arXiv preprint arXiv:2305.12427*, 2023. 1, 3

[49] Thang Vu, Kookhoi Kim, Tung M Luu, Thanh Nguyen, and Chang D Yoo. Softgroup for 3d instance segmentation on point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2708–2717, 2022. 2, 6, 9, 10

[50] Peng-Shuai Wang. Octformer: Octree-based transformers for 3d point clouds. *arXiv preprint arXiv:2305.03045*, 2023. 6, 9, 10

[51] Xiaoyang Wu, Yixing Lao, Li Jiang, Xihui Liu, and Hengshuang Zhao. Point transformer v2: Grouped vector attention and partition-based pooling. *Advances in Neural Information Processing Systems*, 35:33330–33342, 2022. 4, 6, 9, 10

[52] Chenfeng Xu, Shijia Yang, Tomer Galanti, Bichen Wu, Xiangyu Yue, Bohan Zhai, Wei Zhan, Peter Vajda, Kurt Keutzer, and Masayoshi Tomizuka. Image2point: 3d point-cloud understanding with 2d image pretrained models. *arXiv preprint arXiv:2106.04180*, 2021. 3

[53] Jiarui Xu, Sifei Liu, Arash Vahdat, Wonmin Byeon, Xiaolong Wang, and Shalini De Mello. Open-vocabulary panoptic segmentation with text-to-image diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2955–2966, 2023. 1

[54] Karmesh Yadav, Ram Ramrakhya, Santhosh Kumar Ramakrishnan, Theo Gervet, John Turner, Aaron Gokaslan, Noah Maestre, Angel Xuan Chang, Dhruv Batra, Manolis Savva, et al. Habitat-matterport 3d semantics dataset. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4927–4936, 2023. 3

[55] Yu-Qi Yang, Yu-Xiao Guo, Jian-Yu Xiong, Yang Liu, Hao Pan, Peng-Shuai Wang, Xin Tong, and Baining Guo. Swin3d: A pretrained transformer backbone for 3d indoor scene understanding. *arXiv preprint arXiv:2304.06906*, 2023. 6, 9, 10

[56] Renrui Zhang, Ziyu Guo, Wei Zhang, Kunchang Li, Xupeng Miao, Bin Cui, Yu Qiao, Peng Gao, and Hongsheng Li. Pointclip: Point cloud understanding by clip. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8552–8562, 2022. 3

[57] Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip HS Torr, and Vladlen Koltun. Point transformer. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 16259–16268, 2021. 4

[58] Weiguang Zhao, Yuyao Yan, Chaolong Yang, Jianan Ye, Xi Yang, and Kaizhu Huang. Divide and conquer: 3d point cloud instance segmentation with point-wise binarization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 562–571, 2023. 2, 5, 6, 10

[59] Jia Zheng, Junfei Zhang, Jing Li, Rui Tang, Shenghua Gao, and Zihan Zhou. Structured3d: A large photo-realistic dataset for structured 3d modeling. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part IX 16*, pages 519–535. Springer, 2020. 9

[60] Zhisheng Zhong, Jiequan Cui, Yibo Yang, Xiaoyang Wu, Xiaojuan Qi, Xiangyu Zhang, and Jiaya Jia. Understanding imbalanced semantic segmentation through neural collapse. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19550–19560, 2023. 6, 10

[61] Xueyan Zou, Zi-Yi Dou, Jianwei Yang, Zhe Gan, Linjie Li, Chunyuan Li, Xiyang Dai, Harkirat Behl, Jianfeng Wang, Lu Yuan, et al. Generalized decoding for pixel, image, and language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15116–15127, 2023. 5